Comunicación entre procesos

PSP · Batería de ejercicios

> **Proyecto de partida**
>
> Para hacer los ejercicios **tienes que usar el proyecto de partida** `PSP_Comunicacion_Alumnos.zip` que te facilita el profesor. Ya incluye todas las clases que tienes que completar, los ficheros de datos y la configuración necesaria para que los ejercicios funcionen desde IntelliJ. No crees un proyecto nuevo.

Cómo abrir el proyecto en IntelliJ

1. Descomprime `PSP_Comunicacion_Alumnos.zip`.
2. En IntelliJ: **File > Open...** y selecciona la carpeta `PSP_Comunicacion_Alumnos` (la que contiene el fichero `LEEME.txt`).
3. Si aparece el aviso «Project JDK is not defined» o «Setup SDK», pulsa en él y elige tu JDK (versión 11 o superior).

Estructura del proyecto

Carpeta	Contenido
`src/PSP/`	Las clases de los ejercicios (`Ejercicio01` a `Ejercicio15`) y los hijos (`HijoInverso`, `HijoCalculadora`, `HijoPrimos`, `HijoAdivina`). Cada una tiene un `// TODO` donde escribir tu solución.
`datos/`	Ficheros de datos: `notas.csv` (ejercicio 11) y `texto.txt` (ejercicios 5 y 13).
`bin/`	Se crea al compilar. **No cambies esta carpeta.**

Importante

- El proyecto está configurado para compilar en la carpeta `bin`. Los ejercicios que lanzan otro programa Java (9, 10, 14 y 15) usan `java -cp bin PSP.HijoXxx`, así que funcionan ejecutándolos directamente desde IntelliJ.
- IntelliJ ejecuta los programas desde la raíz del proyecto, así que las rutas `datos/notas.csv` y `datos/texto.txt` funcionan tal cual.
- Para los ejercicios que reciben argumentos (`args`), edita la configuración de ejecución: **Run > Edit Configurations... > Program arguments**. Ya hay una preparada para el ejercicio 5: «Ejercicio05 (gato datos/texto.txt)».
- Los ejercicios que piden datos por teclado (`Scanner`) se responden en la consola de IntelliJ.

Alternativa desde la terminal, situándote en la carpeta del proyecto:

```
javac -d bin src/PSP/*.java
java -cp bin PSP.Ejercicio01
```

Normas

- No cambies el paquete PSP ni los nombres de las clases: los padres lanzan a los hijos por su nombre.
- Los comandos son de macOS y funcionan igual en Linux.
- No uses expresiones lambda.
- Responde por escrito a las preguntas marcadas como **Reflexión**.

Recordatorio

Quiero...	Uso
Leer la salida del hijo	`getInputStream()` + `BufferedReader`
Leer los errores del hijo	`getErrorStream()` o `redirectErrorStream(true)`
Enviar datos y terminar	`getOutputStream()` + `PrintWriter` + `close()`
Enviar datos y seguir conversando	`getOutputStream()` + `PrintWriter` + `flush()`
Redirigir a ficheros	`redirectInput()`, `redirectOutput()`, `Redirect.appendTo()`
Encadenar procesos	`ProcessBuilder.startPipeline(lista)`

> **Regla de oro**
>
> Primero se lee la salida del hijo y **después** se llama a `waitFor()`. Si se hace al revés, el programa puede quedarse bloqueado cuando el hijo escribe mucho.

Ficheros de datos

Ya están en la carpeta `datos` del proyecto. Este es su contenido, como referencia:

datos/notas.csv

```
Lucía,7.5
Marcos,9.2
Ana,6.8
Javier,8.4
Sara,9.7
Pablo,5.1
Elena,8.9
```

datos/texto.txt

```
El gato come. El perro come.
El gato duerme y el perro ladra.
Un gato, un perro y un pez.
```

Nivel 1 · Leer la salida del hijo

Ejercicio 1. Eco

Escribe un programa `Ejercicio01` que lance el comando `echo Hola desde el hijo`, lea su salida y la muestre con el prefijo `[HIJO]` . Al final, muestra el código de salida.

Métodos: `getInputStream()`, `BufferedReader`, `readLine()`, `waitFor()`

Reflexión: ¿Por qué el texto aparece con el prefijo `[HIJO]` y no tal cual lo escribe `echo`? ¿Quién lo muestra en pantalla realmente?

Ejercicio 2. ¿Quién y dónde?

Escribe un programa `Ejercicio02` que lance los comandos `whoami` (usuario actual) y `hostname` (nombre del equipo) y, con lo que devuelvan, componga y muestre la frase:

```
Usuario <usuario> en el equipo <equipo>
```

Crea un método `ejecutar(String comando)` que lance un comando y devuelva la primera línea de su salida.

Métodos: `getInputStream()`, `readLine()`, `waitFor()`

Ejercicio 3. Saludo según la hora

Escribe un programa `Ejercicio03` que obtenga la hora actual lanzando `date +%H` (devuelve la hora de 00 a 23), la convierta a `int` y salude según la hora:

- de 6 a 13: «Buenos días»;
- de 14 a 20: «Buenas tardes»;
- el resto: «Buenas noches».

Para poder probar todos los casos, si se pasa una hora por `args`, usa esa en lugar de lanzar `date`.

Reflexión: En la terminal escribirías `date "+%H"` con comillas. ¿Por qué en `ProcessBuilder` no hacen falta?

Ejercicio 4. Los 5 más grandes

Escribe un programa `Ejercicio04` que lance `ls -S <carpeta>` (lista los ficheros ordenados por tamaño, de mayor a menor) y muestre **solo los 5 primeros**, numerados. La carpeta se recibe por `args` (por defecto, `/usr/bin`).

Cuando hayas leído 5 líneas, deja de leer. Si el hijo sigue vivo, termínalo con `destroy()`. Muestra al final el código de salida.

Reflexión: ¿Por qué conviene terminar el hijo si ya no vas a leer su salida? ¿Qué podría pasarle si escribiera mucho y nadie lo leyera?

Nivel 2 · Errores y códigos de salida

Ejercicio 5. Buscador con grep

Escribe un programa `Ejercicio05` que reciba por `args` una palabra y un fichero, y lance `grep -n <palabra> <fichero>`. `grep -n` devuelve las líneas que contienen la palabra precedidas de su número, por ejemplo `3:Un gato, un perro y un pez.`

Muestra cada coincidencia como `Línea 3: Un gato, un perro y un pez.` y, según el código de salida de `grep`:

Código	Significado	Mensaje
0	Encontrado	«"gato" aparece en N líneas»
1	No encontrado	«"caballo" no aparece en texto.txt»
2	Error (p. ej., el fichero no existe)	El mensaje de error de `grep`

Usa el fichero `datos/texto.txt` para las pruebas.

Ejercicio 6. Revisor de comandos

Escribe un programa `Ejercicio06` que ejecute esta lista de comandos, uno detrás de otro:

```
date
ls carpetaQueNoExiste
comandoInventado
cat ficheroQueNoExiste.txt
whoami
```

Para cada uno, muestra una línea de informe:

- `[OK]` si termina con código 0;
- `[FALLO n]` con el código y la primera línea del mensaje de error si termina con otro código;
- `[NO EXISTE]` si el comando no se puede lanzar.

Al final, muestra cuántos han ido bien. Usa `redirectErrorStream(true)` para leer los errores por el mismo flujo que la salida.

Reflexión: ¿Por qué `comandoInventado` no tiene código de salida? ¿En qué se diferencia de `ls carpetaQueNoExiste`?

Nivel 3 · Enviar datos al hijo

Ejercicio 7. Calculadora con bc

`bc` es una calculadora de la terminal: lee expresiones de su entrada y escribe los resultados. Pruébala antes en la terminal: escribe `bc -l`, después `2^10` e Intro, y sal con Ctrl+D.

Escribe un programa `Ejercicio07` que pida una expresión con `Scanner`, la envíe a `bc -l`, lea el resultado y lo muestre. Si `bc` no puede calcularla, muestra su mensaje de error y el código de salida.

Prueba con: `2^10`, `355/113`, `sqrt(2)`, `3 +* 4` y `1/0`.

Reflexión: ¿Qué pasa si no cierras el `PrintWriter` después de enviar la expresión? ¿Qué equivalencia tiene `close()` con lo que hiciste en la terminal?

Ejercicio 8. Cifrado César con tr

El comando `tr 'A-Za-z' 'N-ZA-Mn-za-m'` aplica el cifrado ROT13: cambia cada letra por la que está 13 posiciones más adelante. Como el alfabeto inglés tiene 26 letras, aplicarlo dos veces devuelve el texto original.

Escribe un programa `Ejercicio08` con un método `rot13(String texto)` que envíe el texto a `tr` y devuelva lo que responde. El programa debe:

1. pedir un texto al usuario;
2. cifrarlo con un proceso `tr` y mostrarlo;
3. enviar el texto cifrado a **otro** proceso `tr` y mostrar el resultado;
4. comprobar si coincide con el original.

Reflexión: Prueba con un texto con «ñ» o con tildes. ¿Qué ocurre con esas letras? ¿Por qué?

Ejercicio 9. Hijo inversor

Crea dos clases:

- `HijoInverso`: lee líneas de su entrada y, por cada una, escribe la línea al revés. Termina cuando se cierra su entrada.
- `Ejercicio09` (padre): lanza el hijo y pide frases al usuario hasta que escriba `salir`. Envía cada frase al hijo, muestra la respuesta e indica si la frase es un **palíndromo** (sin tener en cuenta espacios ni mayúsculas).

Prueba con «anita lava la tina». Ejecuta el padre directamente desde IntelliJ: el proyecto ya está preparado para que encuentre al hijo.

Reflexión: En los ejercicios 7 y 8 se cerraba el flujo después de enviar. ¿Por qué aquí se usa `flush()` en lugar de `close()`?

Ejercicio 10. Hijo calculadora

Crea dos clases que se comuniquen con un **protocolo** sencillo:

- `HijoCalculadora`: lee líneas con el formato `número operador número` (por ejemplo, `3 + 4`) y responde con el resultado. Admite `+ - * /`. Si la operación no es válida (formato incorrecto, operador desconocido o división entre cero), responde `ERROR`. Termina al recibir `fin`.
- `Ejercicio10` (padre): envía al hijo, una a una, las operaciones de un array, muestra cada resultado y, al final, cuántas no eran válidas.

```
String[] operaciones = {"3 + 4", "10 / 4", "6 * 7", "8 / 0", "hola + 2", "5 ^ 2", "20 - 25"};
```

Reflexión: ¿Qué es un protocolo de comunicación? Enumera las reglas del protocolo que usan este padre y este hijo.

Nivel 4 · Ficheros y tuberías

Ejercicio 11. Ordenar notas

El fichero `datos/notas.csv` contiene una línea por alumno con el formato `nombre,nota`. Escribe un programa `Ejercicio11` que:

1. ordene el fichero por nota, de mayor a menor, con `sort -t, -k2 -rn`, leyendo de `datos/notas.csv` y escribiendo en `datos/notas_ordenadas.csv`, **sin leer ni escribir los datos desde Java**;
2. después, lea el fichero ordenado desde Java y muestre el podio (oro, plata y bronce).

Opción de sort	Significado
`-t,`	El separador de campos es la coma
`-k2`	Ordenar por el segundo campo
`-rn`	Orden numérico (`n`) inverso (`r`)

Reflexión: Escribe el comando de terminal equivalente al paso 1. ¿Qué ocurre si quitas la `n` de `-rn` y una nota es `10`?

Ejercicio 12. Diario de actividad

Escribe un programa `Ejercicio12` que, cada vez que se ejecute, **añada** al fichero `diario.log`:

1. una línea de cabecera con la fecha y la hora, usando `date` con este formato: `+=== %d/%m/%Y %H:%M:%S ===`;
2. a continuación, la salida de `ls -l`.

Después, lee `diario.log` desde Java y muestra cuántas entradas tiene (líneas que empiezan por `===`) y la última fecha añadida. Ejecútalo varias veces.

Ejercicio 13. Palabras de un texto con una tubería

Escribe un programa `Ejercicio13` que cuente cuántas **palabras distintas** tiene `datos/texto.txt`, usando una tubería de tres procesos equivalente a:

```
tr -cs 'A-Za-z' '\n' < datos/texto.txt | sort -u | wc -l
```

Proceso	Qué hace
`tr -cs 'A-Za-z' '\n'`	Cambia todo lo que no es una letra por un salto de línea: deja una palabra por línea
`sort -u`	Ordena las palabras y elimina las repetidas
`wc -l`	Cuenta las líneas, es decir, las palabras distintas

Antes de programarlo, construye la tubería en la terminal pieza a pieza para ver qué aporta cada una.

Ampliación: muestra las 5 palabras más frecuentes, sin distinguir mayúsculas de minúsculas:

```
tr -cs 'A-Za-z' '\n' < datos/texto.txt | tr 'A-Z' 'a-z' | sort | uniq -c | sort -rn | head -5
```

Reflexión: El resultado de la parte básica es 11, pero el texto solo tiene 9 palabras diferentes si no se distinguen mayúsculas. ¿Por qué? ¿De qué proceso lee el padre y por qué solo de ese?

Nivel 5 · Retos

Ejercicio 14. Contadores de primos en paralelo

Crea dos clases:

- `HijoPrimos`: recibe por `args` dos números, `desde` y `hasta`, cuenta los números primos de ese rango y escribe **solo** el resultado.
- `Ejercicio14` (padre): reparte el rango del 1 al 9.000.000 en tres hijos (1-3.000.000, 3.000.001-6.000.000 y 6.000.001-9.000.000), recoge los tres resultados y muestra el total.

Hazlo de dos formas y mide el tiempo de cada una con `System.currentTimeMillis()`:

- **Secuencial:** lanza un hijo, lee su resultado, lanza el siguiente...
- **Paralela:** lanza los tres hijos y después recoge los resultados.

Métodos: `getInputStream()`, `Integer.parseInt()`, `waitFor()`

Reflexión: ¿Cuántas veces más rápida es la versión paralela? ¿Por qué no es exactamente 3 veces? ¿Por qué los tres hijos no encuentran el mismo número de primos?

Ejercicio 15. Adivina el número

Crea dos clases:

- `HijoAdivina`: elige un número secreto aleatorio del 1 al 100 (o el que reciba por `args`, para pruebas). Lee intentos de su entrada y responde con una palabra: `mayor` (el secreto es mayor), `menor` (el secreto es menor) o `acertado`, y entonces termina.
- `Ejercicio15` (padre): juega **solo** contra el hijo usando **búsqueda binaria**: prueba siempre el punto medio del intervalo posible y lo reduce según la respuesta. Muestra cada intento y, al final, cuántos ha necesitado.

Reflexión: ¿Cuántos intentos necesita el padre como máximo para adivinar un número del 1 al 100? ¿Y del 1 al 1.000.000?